

ASP.NET MVC Web API Project

Choose any kind of item you want and like to manage, and name the types accordingly.

URL	Method	Authorize	Description	Request body	Response body
/api/item	GET	User/Admin	Get all user's items		list of items
/api/item/{id}	GET	User/Admin	Get a user's item by ID		item
/api/item	POST	User/Admin	Add a new item to user	item	item location
/api/item/{id}	PUT	User/Admin	Update user's item	item	
/api/item/{id}	DELETE	User/Admin	Delete user's item		
/api/user/me	GET	User/Admin	Get my user details		user
/api/user	GET	Admin	Get all users		list of users
/api/user	POST	Admin	Add a new user	user	user location
/api/User/{id}	PUT	User/Admin	Update user details	user	
/api/user/{id}	DELETE	Admin	Delete user + their items		
/api/login	POST		Login	user	JWT

Server-side notes:

- Only administrators see all users, and can add/delete users.
- Administrator can edit all users, including all properties.
Regular user can only edit their own user details, and cannot make themselves administrator.
- Users can handle only their own items, and cannot see items of other users.
- Both items and users should be saved in a JSON file.
- Both items and users should be accessed via an injected service through an interface, so it would be easy to change it to a DB-based app.
- Add a scoped service for active user, and inject it wherever you need, so the user ID, role, and other details will be available easily.
- Add an extension method of IServiceCollection for an easy registration of all your services.
- Use SignalR to notify a user's other active connections (different tabs/devices) when they add/update/delete items (track user connections in the Hub using OnConnectedAsync / OnDisconnectedAsync, and broadcast only to the same user's connection IDs using Clients.Clients(connectionIds)).
- Each request should be written to a log file, including start date & time, name of controller & action, name of the logged-in user if there is one, and the duration of operation in milliseconds. Use a background worker with a message queue to write logs asynchronously without blocking request threads.

Client-side notes:

- The default page should show the user's items and allow add/update/delete operations.
- If there is no logged-in user (no token saved in local storage, or the token expired), show a login page instead of the default page.
- After add/update/delete items, show only a toast notification indicating the operation was successful. The grid should update only when SignalR notifies that an update happened.
- If the user has administrative privileges, show a link from the items list page to the users list page, and vice versa.

Challenge:

- Replace your custom logging implementation with a popular logging library such as Serilog or log4net. Make sure the log file path is configurable and implement log file rotation to prevent files from growing larger than 50MB.
- Allow users to log in using their Google account.